

Code Explanation

Batch Processing Pipeline Explanation

=====

This code is designed to perform batch processing on customer and order data using Apache Spark. The pipeline involves reading data from source files, cleaning and transforming the data, and writing it to Delta tables.

Overview of the Code

The code is divided into several functions, each responsible for a specific task in the batch processing pipeline. The main function orchestrates the entire process, from creating a Spark session to writing the processed data to Delta tables.

Step 1: Creating a Spark Session

Creating a Spark session is the first step in the batch processing pipeline. This is done using the `create\_spark\_session` function, which returns a Spark session with the app name set to "Customer Order Processing".

```
def create_spark_session():
    return SparkSession.builder
        .appName("Customer Order Processing")
        .getOrCreate()
```

Step 2: Reading Customer and Order Data

The `read\_customer\_data` and `read\_order\_data` functions read customer and order data from CSV files using the Spark session.

```
def read_customer_data(spark, path):
    return spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(path)

def read_order_data(spark, path):
    return spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(path)
```

Step 3: Cleaning Customer and Order Data

The `clean\_customer\_data` and `clean\_order\_data` functions clean the customer and order data by removing nulls and duplicates. The `clean\_order\_data` function also adds a new column "TotalAmount" by multiplying "PricePerUnit" and "Qty".

```
def clean_customer_data(df):
    return df.dropDuplicates(["CustId"])
        .filter(F.col("CustId").isNotNull() &
                F.col("Name").isNotNull() &
                F.col("EmailId").isNotNull() &
```

```

        F.col("Region").isNotNull())

def clean_order_data(df):
    return df.withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates(["OrderId"])
        .filter(F.col("OrderId").isNotNull() &
                F.col("ItemName").isNotNull() &
                F.col("PricePerUnit").isNotNull() &
                F.col("Qty").isNotNull() &
                F.col("Date").isNotNull() &
                F.col("CustId").isNotNull())

```

Step 4: Creating or Updating the Order Summary Table

The `create\_or\_update\_ordersummary` function creates or updates the order summary table using Slowly Changing Dimension (SCD) Type 2. It joins the customer and order data, checks if the table exists, and updates it accordingly.

```

def create_or_update_ordersummary(spark, customer_df, order_df, catalog, schema):
    # Join customer and order data
    joined_df = customer_df.join(order_df, "CustId", "inner")
        .select(
            "CustId",
            "Name",
            "EmailId",
            "Region",
            "OrderId",
            "ItemName",
            "PricePerUnit",
            "Qty",
            "Date",
            "TotalAmount"
        )

    # Check if table exists
    table_path = f"{catalog}.{schema}.ordersummary"
    table_exists = False
    try:
        spark.table(table_path)
        table_exists = True
    except:
        pass

    if not table_exists:
        # First time load - create table with SCD Type 2 columns
        result_df = joined_df
            .withColumn("IsActive", F.lit(True))
            .withColumn("StartDate", F.current_timestamp())
            .withColumn("EndDate", F.lit(None).cast("timestamp"))

        result_df.write.format("delta")
            .mode("overwrite")
            .option("overwriteSchema", "true")
            .saveAsTable(table_path)
    else:

```

```

# Get existing data
delta_table = DeltaTable.forName(spark, table_path)
existing_df = delta_table.toDF()
active_records = existing_df.filter(F.col("IsActive") == True)

# Find records with changes in customer data
changed_customers = customer_df
    .join(
        active_records.select("CustId", "Name", "EmailId", "
Region").distinct(),
        "CustId",
        "inner"
    )
    .filter(
        (F.col("Name") != active_records["Name"]) |
        (F.col("EmailId") != active_records["EmailId"]) |
        (F.col("Region") != active_records["Region"])
    )
    .select("CustId")
    .distinct()

# If there are changes, update the SCD Type 2 table
if changed_customers.count() > 0:
    # Create new records for changed customers
    new_records = joined_df
        .join(changed_customers, "CustId", "inner")
        .withColumn("IsActive", F.lit(True))
        .withColumn("StartDate", F.current_timestamp())
        .withColumn("EndDate", F.lit(None).cast("timestamp"))

    # Update existing records (make them inactive)
    delta_table.alias("target")
        .merge(
            changed_customers.alias("source"),
            "target.CustId = source.CustId AND target.IsActive = true"
        )
        .whenMatched()
        .updateExpr({
            "IsActive": "false",
            "EndDate": "current_timestamp()"
        })
        .execute()

    # Insert new records
    new_records.write.format("delta")
        .mode("append")
        .saveAsTable(table_path)

```

Step 5: Creating or Updating the Customer Aggregate Spend Table

The `create\_or\_update\_customeraggregatespend` function creates or updates the customer aggregate spend table by aggregating data from the order summary table.

```
def create_or_update_customeraggregatespend(spark, catalog, schema):
    table_path = f"{catalog}.{schema}.customeraggregatespend"
    ordersummary_path = f"{catalog}.{schema}.ordersummary"

    # Read from ordersummary table
    ordersummary_df = spark.table(ordersummary_path)

    # Aggregate data
    aggregated_df = ordersummary_df
        .filter(F.col("IsActive") == True)
        .groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))

    # Write to customeraggregatespend table
    aggregated_df.write.format("delta")
        .mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable(table_path)
```

Step 6: Running the Batch Processing Pipeline

The `main` function runs the batch processing pipeline by calling the various functions in the correct order.

```
def main():
    # Define paths and settings
    customer_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata"
    order_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata"
    catalog = "gen_ai_poc_databrickscoe"
    schema = "sdlc_wizard"

    # Create Spark session
    spark = create_spark_session()

    # Read source data
    customer_df = read_customer_data(spark, customer_path)
    order_df = read_order_data(spark, order_path)

    # Clean data
    clean_customer_df = clean_customer_data(customer_df)
    clean_order_df = clean_order_data(order_df)

    # Write cleaned data to delta tables
    clean_customer_df.write.format("delta")
        .mode("overwrite")
        .saveAsTable(f"{catalog}.{schema}.customer")

    clean_order_df.write.format("delta")
        .mode("overwrite")
        .saveAsTable(f"{catalog}.{schema}.order")

    # Create or update ordersummary table (SCD Type 2)
    create_or_update_ordersummary(spark, clean_customer_df, clean_order_df, catalog, schema)

    # Create or update customeraggregatespend table
```

```
    create_or_update_customeraggregatespend(spark, catalog, schema)
if __name__ == "__main__":
    main()
```